

Database Design Assumptions and Key Decisions

1. We use separate Category and Address tables to avoid duplication of data in the event where two customers may share the same address.
2. An inventory_movements table is included for keeping a log of stock changes such as regular sales, returns, restocking, and adjustments.
3. The database currently supports only CAD and USD. However, this check constraint can easily be changed in the future if desired.
4. The policy documents table stores metadata for documents that we want accessible to the RAG chatbot. From there we can automate the updating of the vector database based on the contents of this table i.e. a python script.
5. For payment tracking, our database is using two separate tables, accounts_payable and accounts_receivable.
6. A separate refunds table is included, as one of the main goals of the database redesign was simplifying the refund process. This will allow us to have specific metadata and status tracking for each refund.
7. An order_items table is used so that orders can support multiple products / quantities, and this can be easily queried.
8. A carriers table is used for storing a name, description, and importantly a tracking url template for each carrier that we may ship with.
9. For payments we decided to disjoint our payment types which allows us to make certain types of payments follow a specific rule set. Having the payment types disjointed also allows for each type to be mutually exclusive.
10. Another disjointed subtype we implemented was for payees which can either be a supplier for the business or a carrier we contracted. A payee can either be one or the other, not both.
11. Using accounts as a supertype, we choose to have staff and customer overlap for the reason of a staff member could be a customer, and a customer could be a staff member
12. We decided that it made sense for payments and orders should have an aggregate relationship because one cannot exist without the other

Proposal revisions

1. High volume data source: Customer transactions, as well as read operations for product details
2. Docker use justification: Docker makes consistent and reproducible deployments easier, especially if we are deploying on a cloud environment.